

Stacks - 2

TABLE OF CONTENTS

1. Nearest Smaller Element [on L.H.S & on R.H.S]
2. Nearest Greater Element (NGE) - [on L.H.S & on R.H.S]
3. Index of Nearest Smaller
4. Largest Rectangle in Histogram
5. Sum of (max-min) for all Subarrays



Notes





Nearest Smaller Element on LEFT

< Question > : Given an array of size N. For every index i, find the nearest element which is smaller than ith element on left side.

$1 \leq N \leq 10^5$

Example 1 :

arr : [4 5 2 10 3 2]
 0 1 2 3 4 5

Ans → -1 4 -1 2 2 -1

Example 2 :

arr : [4 6 10 11 7 8 3 5]
 0 1 2 3 4 5 6 7

Ans → -1 4 6 10 6 7 -1 3

3

[4, 6, 13, 12, 11, 7, 8, 3, 5]
-1 4 6 6 6 6 7 -1 3



BF Idea

```
ans[N] = {-1}
for(i=0 ; i<N ; i++){
    for(j=i-1 ; j>=0 ; j--){
        if(A[j] < A[i]){
            ans[i] = A[j]
            break
        }
    }
}
return ans
```

TC → $O(N^2)$
SC → $O(1)$



- **Idea for optimisation** (maintain some kind of history) ^{as there cannot be a single ans}

Say we travel from L to R & stand at $A[i]$. Now, this $A[i]$ can be answer for some ele in future. We basically have to maintain history, hence stack data structure can be useful.

⇒ So the intuition is while traversing from left to right
Stack top will have the next smaller elem → if it greater than pop it off

(point is not about)
when to add → add all

point is when to
remove

at 5 no element, but 5 can be a potential
ans so insert 5
for next

arr[]: [5 2 8 10 6 1 7 15]
 0 1 2 3 4 5 6 7
 -1 -1 2 8 2 -1 1 7

① → if (stack.isEmpty())
→ insert in stack
→ ans[i] = -1
}



at 2 any element which have 5 as
the ans will definitely use 2 because
 $2 < 5$ & 2 is more closer to
any element that is going to come than
5 so remove 5

② if ($A[i] < \text{stack.peek()}$) {

→ empty stack till element $< A[i]$
not found

if stack is empty {

→ ans[i] = -1

} else {
ans[i] = stack.peek()
}

→ insert $A[i]$ in stack
}



Idea for optimisation

at 8 $\textcircled{3}$ if ($a[i] > \text{stack.peak}()$) {
 $\text{ans}[i] = \text{stack.peak}()$
 $\rightarrow$ insert 8 in stack ?? $\rightarrow$ yes
}

so stack $\rightarrow$

8
2

arr[] : [5 2 8 10 6 1 7 15]
 0 1 2 3 4 5 6 7
 -1 -1 2 8 2 -1 1 7

any ele greater than 5,
 will be greater than 2.
 so we can simply
 remove 5 from stk.

don't remove 2,
 only insert 8.
 what if an ele comes which
 is < 8 but > 2 ? say 10

at $i = 10$ $\rightarrow$ Cond 3 $\text{ans}[i] = 8$

at $i = 6 \rightarrow$ Cond 2
 $\text{ans}[i] = 6$

at $i = 1 \rightarrow$ Cond 2
 $\text{ans}[i] =$

6
10
8
2

<u>pk</u>	1	6 > 4	10 > 6	11 > 10	7 > 11xx		
	4	6	10	11	7	8	3
							5
Stack is empty					→ keep on removing till top < 7		
Stack	4						
		6 4	10 6 4	11 10 6 4	7 11 10 6 4	8 7 6 4	3 8 7 6 4
							5 3
<u>ans</u>	-1	4	6	6	6	7	-1
							3

i = 0; i < n; i++

num = arr[i]

while (!stack.isEmpty() &&

| num < stack.peek()) {

| stack.pop();

}

ans[i] = stack.isEmpty() ? -1 : stack.peek();

stack.push(arr[i]);

{ 0 1 2 3 4 5 6 7 }

4 6 10 11 7 8 3 5

↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑

-1 4 6 10 6 7 -1 3

(0) (1) (2) (1) (4) (6)

Stack is always maintained in ↑ order.

5
3



</> Pseudo-Code

ans[N] = {-1} // set values of all
ele as -1

Stack <int> S;

for (i = 0; i < N; i++) {

while (st.size() > 0 &&
st.top() >= A[i]) {
st.pop();

if (st.size() > 0) {
ans[i] = st.top();

st.push(A[i])

}

i

5
3

A[i] > st.top()

ans[i] = st.top()

A[i] <= st.top()

Keep on popping from
stack till we
find lesser ele or
st. becomes
empty.

↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
{ 0 1 2 3 4 5 6 7 }
4 6 10 11 7 8 3 5
-1 4 6 10 6 7 -1 3

* How many times will we touch any ele in worst case?

Not more than 2 times.

Once while pushing on stack & once while popping
it.

* Rather have indices as the answer ✓

HW:

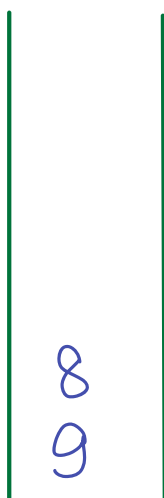
0	1	2	3	4	5	6	7	8	9
4	7	-9	5	-6	-1	7	5	2	10

This helps us deal with ve- elements
as well.



Nearest Greater Element on LEFT

-1	-1	-1	9	9	-1	9	7	5	9
0	1	2	3	4	5	6	7	8	9
4	7	9	5	6	9	7	5	2	8



if (st.top() <= A[i])
pop()

if (st.top() > A[i])
make it as ans

insert A[i]

just push 5, don't
remove 7.
what if some ele is >5 but
<7?

while (stack.isEmpty()) &&

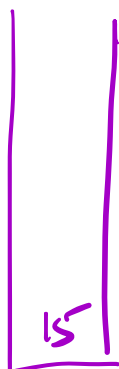
arr[i] >= stack.peek()

stack.pop()

arr[]: [5 2 8 10 6 1 7 15]
-1 5 -1 -1 10 6 10 -1

ans[i] = stack.isEmpty() ? -1 : stack.peek()

stack.push(ans[i])





Nearest Smaller Element on RIGHT

(travel from right to left)

→ why ⇒ because when we travel in that manner we already know the ans at right side

arr : [7 9 3 5 8 11 6]



arr : [9 7 3 5 4 2 6 1 8]

0 1 2 3 4 5 6 7 8

</> Pseudo-

ans[N] = {-1} // set values of all
ele as -1

Stack<int> S;

for (i = n-1; i ≥ 0; i--) {

while (st.size() > 0 &&
st.top() ≥ A[i]) {
st.pop();
}

if (st.size() > 0) {
ans[i] = st.top();
}

st.push(A[i])

}

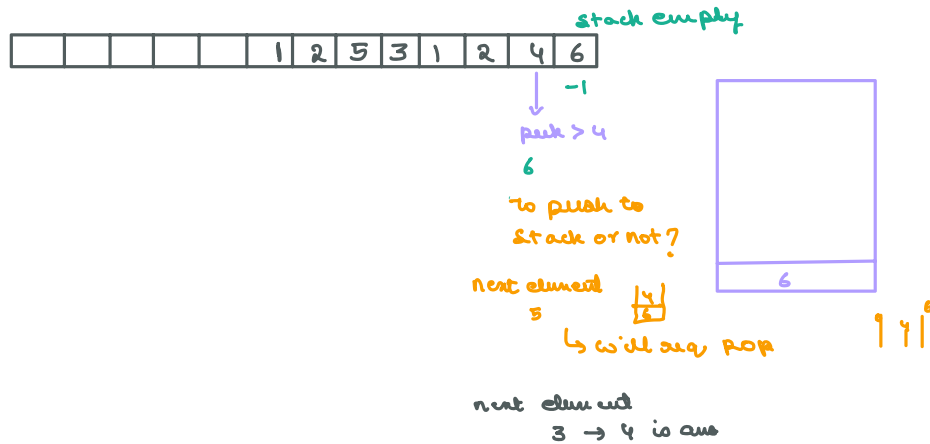
i

5
3

↓ ↓ ↓ ~
0 1 2
4 6 10 1
-1 4 6



Nearest Greater Element on RIGHT



So question is not about to push it in the stack or not
 ↳ push will be req when to pop previous to them is question

↳ when peek < current we push till we don't find next larger

such that all the element of stack is monotonic

Break → 8:32 AM

1 2 3 4 5 6 7 8 9 10 11 12

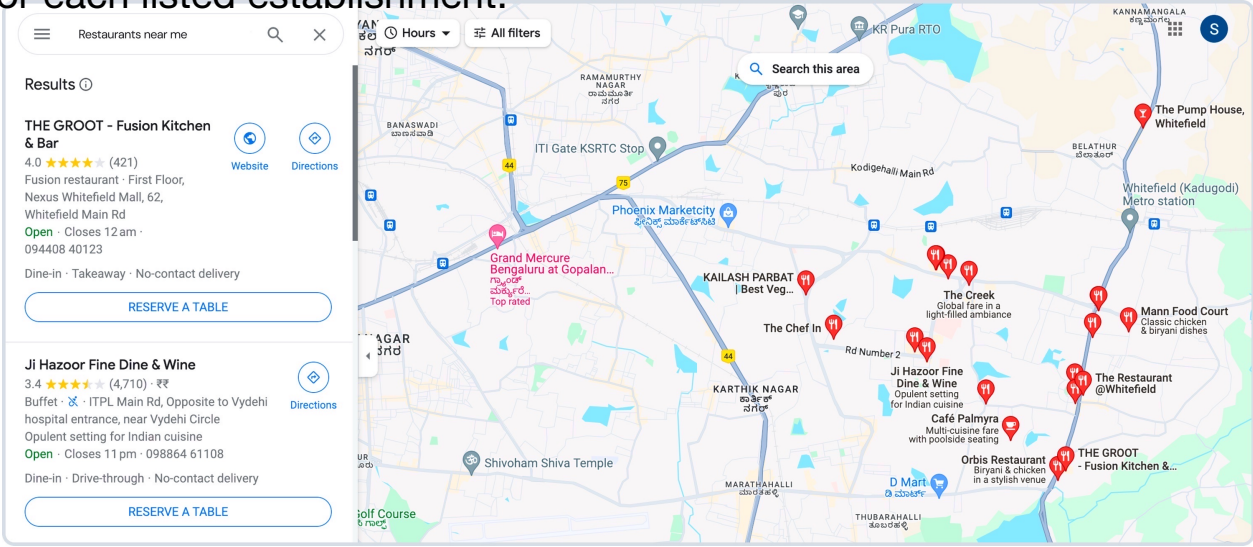
- A person uses Google Maps to find the nearest restaurants and picks one based on it's proximity. Unfortunately, after visiting, they realised that the restaurant didn't meet their expectations.

Task

Let's break it down with a simple example. You have a list of restaurants and their ratings. For each restaurant, we're going to find the next restaurant to the right on the list that's not just close but also has a higher rating than the current one. If there's no better option on the list, we'll say there's none available.

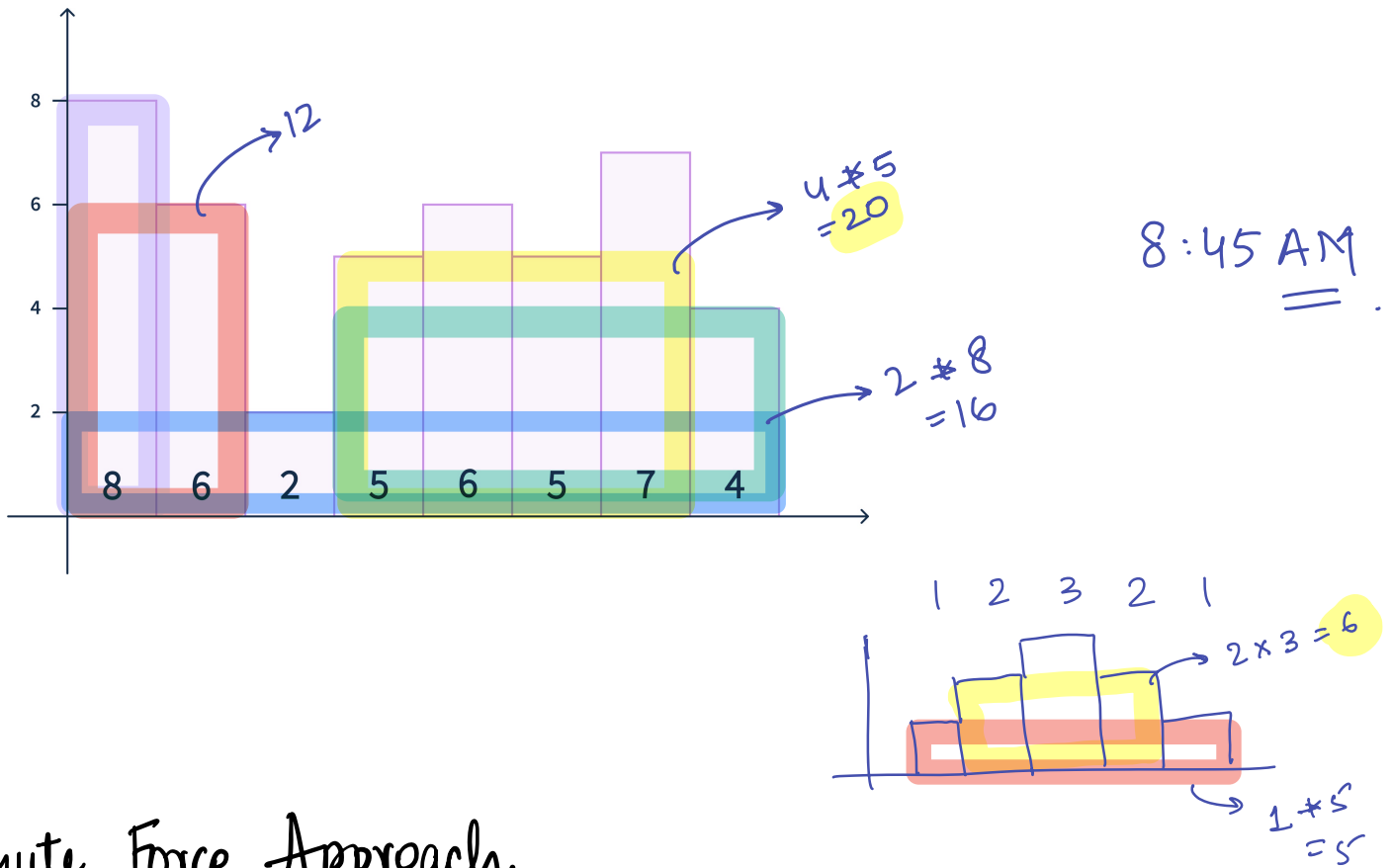
Problem

Given a sequence of restaurants listed on Google Maps with their ratings, create a tool that helps users discover the rating of the next higher-rated restaurant to the right for each listed establishment.



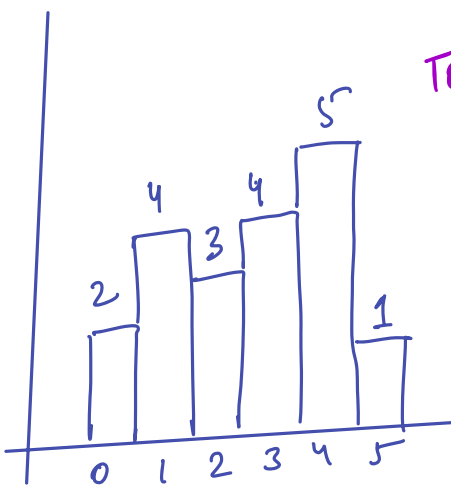


< Question > : Find the largest area of rectangle (formed by continuous bars) in histogram.



Brute Force Approach

Take 2 bars at a time & take area between them.



Total n^2 pairs have to be created.

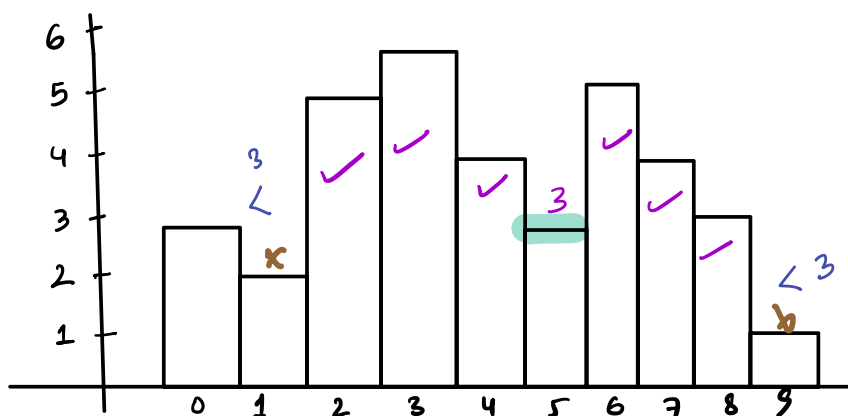
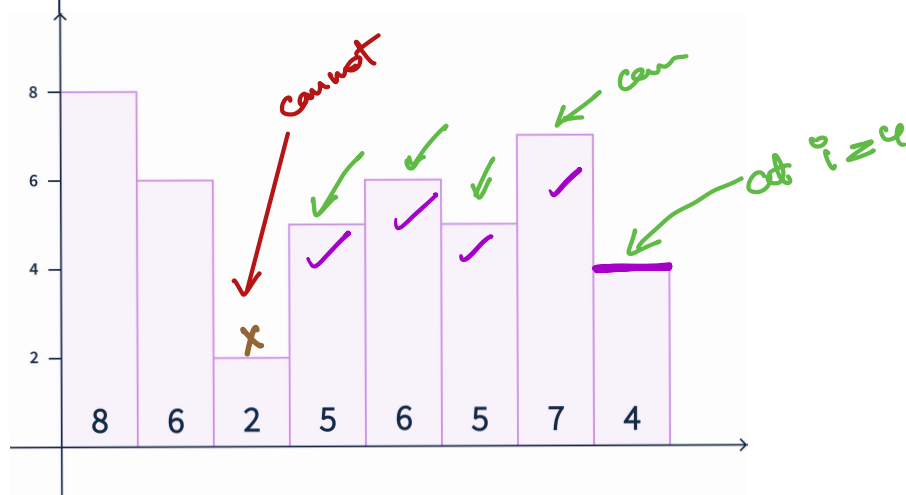
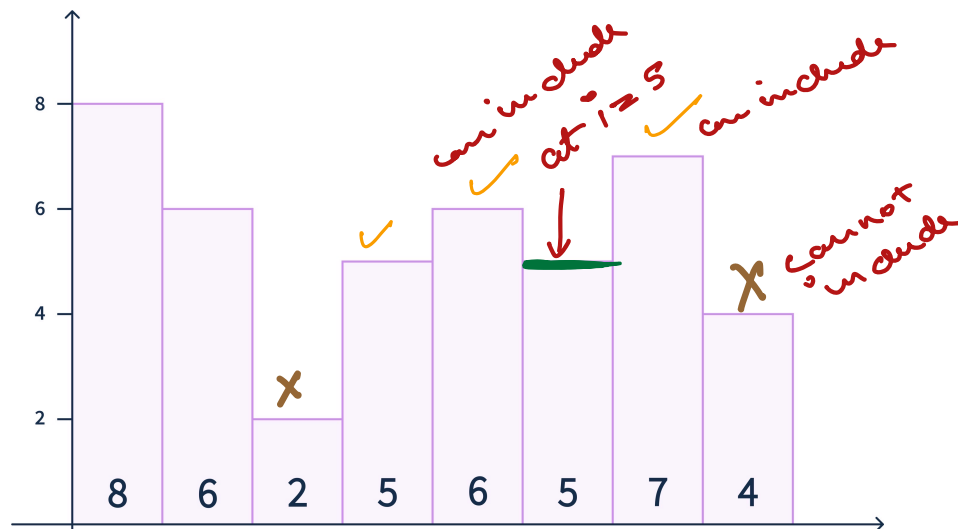
s	e	H	W	Area
0	0	2	1	2
0	1	2	2	4
0	2	2	3	6
0	3	2	4	8
0	4	2	5	10
...	...	...	...	...
1	1	4	1	4
1	2	3	2	6
...	...	...	...	...



Observation :

Say you to include a building, Can we increase height more than it? It should be atleast that much.

What should be the width?



Nearest smaller on left of 3, & on right of 3.

$$x_1 = 1$$

$$x_2 = 9$$

$$x_2 - x_1 - 1$$

$$9 - 1 - 1 = 7 \text{ (width)}$$

ans = $-\infty$

left[] // nearest smaller ele's index on left $\rightarrow O(N)$

right[] // nearest smaller ele's index on right $\rightarrow O(N)$

Traverse through array } $O(N)$

$n1 = \text{left}[i];$

$n2 = \text{right}[i];$

$W = n2 - n1 - 1$

$H = A[i]$

$\text{area} = H * W$

$\text{area} \geq \text{area} \leq -1 ? H : \text{area}$

$\text{ans} = \max(\text{ans}, \text{area})$

$\rightarrow$ why min because

$\text{TC} : O(N)$
 $\text{SC} : O(N)$



$$\begin{matrix} n1 \geq -1 \\ n2 \geq -1 \end{matrix} \} \rightarrow w = -1 - (-1) - 1) \geq -1$$

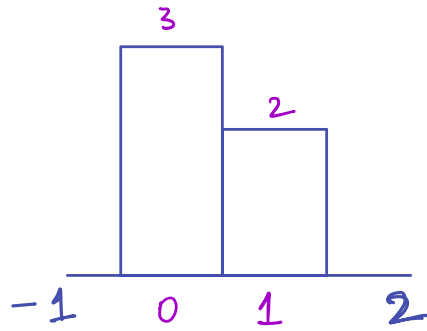
hence

area can be

$$-1 \times h = h$$

but in this case
width is 1 & h

Edge Case



smaller
on left

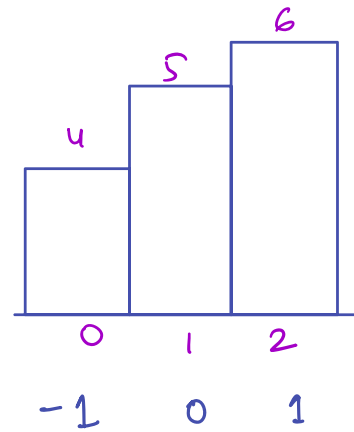
-1 -1

smaller on
right

1 2

$$\begin{aligned} H &= 3 \\ W &= 1 - (-1) - 1 \\ &= 1 \\ \text{area} &= 3 \end{aligned}$$

$$\begin{aligned} H &= 2 \\ W &= 2 - (-1) - 1 \\ &= 2 + 1 - 1 \\ &= 2 \\ \text{area} &= 2 * 2 \\ &= 4 \end{aligned}$$



$$\begin{aligned} W &= 3 - (-1) - 1 \\ &= 3 \\ W &= 3 - 0 - 1 \\ &= 2 \\ W &= 3 - 1 - 1 \\ &= 1 \end{aligned}$$

If no smaller is found on right, then keep n as the answer.



< Question > : Find sum of max for all subarrays.

$$1 \leq N \leq 10^5$$

arr : [1 2 3]
0 1 2

	max
[1]	1
[1,2]	2
[1,2,3]	3
[2]	2
[2,3]	3
[3]	3
	14

another way of looking at the same:

$$1*1 + 2*2 + 3*3 = 14$$

Instead →

an ele is max in how many subarrays?

{ 4 5 7 }

$$\{4\} = 4$$

$$\{5\} = 5$$

$$\{7\} = 7$$

$$\{4\ 5\} = 5$$

$$\{5\ 7\} = 7$$

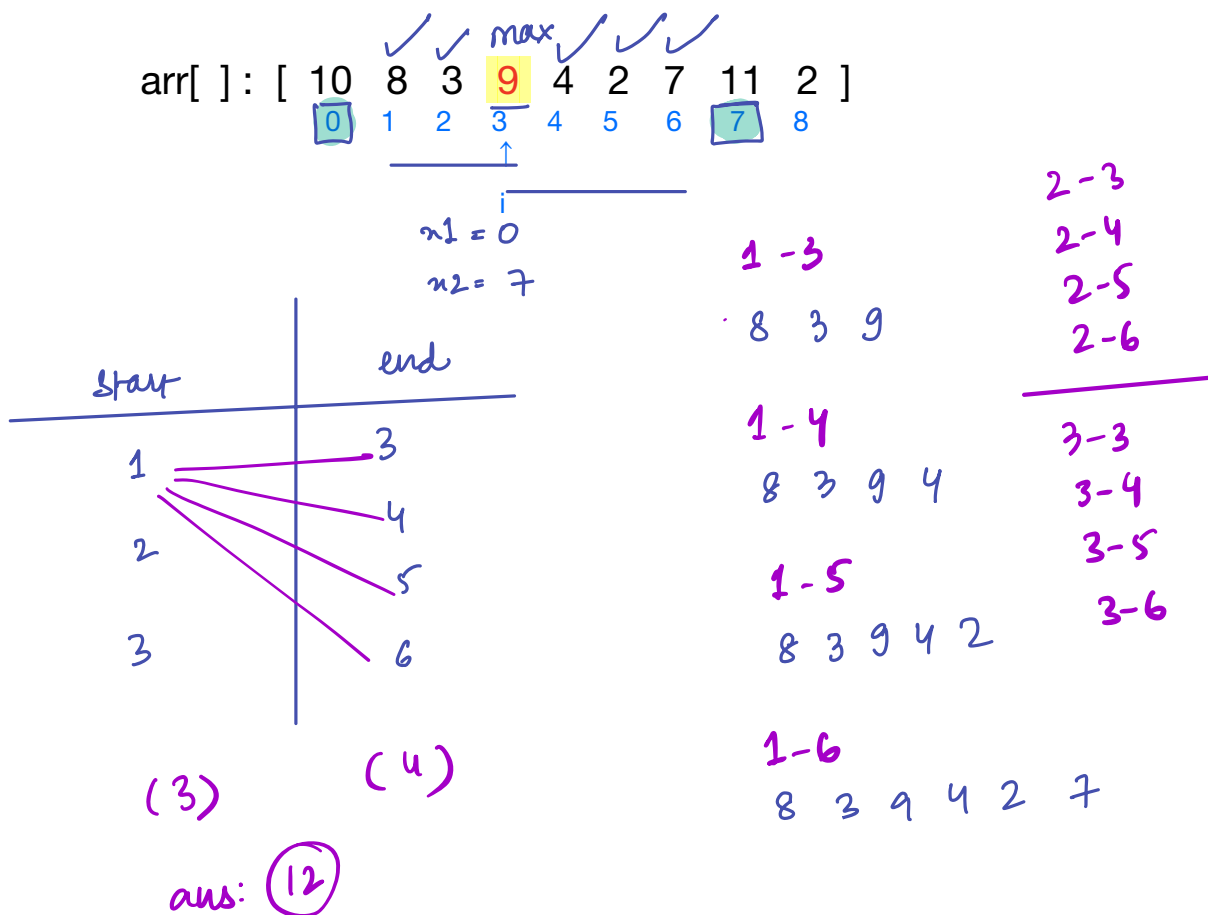
$$\{4\ 5\ 7\} = 7$$

$$1 \times 4 + 2 \times 5 + 3 \times 7 = \underline{\hspace{2cm}}$$



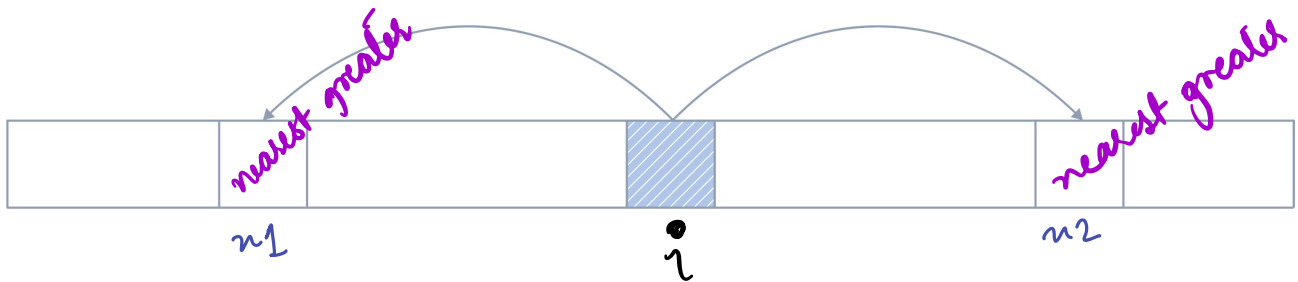
• Optimisation

< **Question** > : In how many subarrays, ith element is the maximum element?





- Generalisation



total subarrays where $arr(i)$ is max $\rightarrow$

$$[a \ b] \\ b - a + 1$$

<i>start</i>	<i>end</i>
$[n1+1 \quad i]$	$[i \quad n2-1]$
$i - (n1+1) + 1$	$n2 - i + 1$
$i - n1 - 1 + 1$	$n2 - i$
$i - n1$	



</> Code

$ngl[] = \{-1\}$ // nearest greater on left index $- O(N)$

$ngr[] = \{N\}$ // nearest greater on right index $- O(N)$

for ($i = 0$; $i < N$; $i++$) { $- O(N)$

$n1 = ngl[i]$

$n2 = ngr[i]$

$start = i - n1$

$end = n2 - i$

$sub = start * end$

$contri = A[i] * sub$

$ans += contri$

$TC \rightarrow O(N)$

$SC \rightarrow O(N)$

}

0	1	2	3
4	3	9	7

left

right

$n1: -1$

$n2: 4$

$start: 2 - (-1)$
 $= 3$

$end: 4 - 2$
 $= 2$